

DRIP: An R Package for Jump Regression and Image Analysis

by Yicheng Kang

Abstract Jump regression performs nonparametric regression analysis where the regression function can be discontinuous. Because of its ability to conduct jump detection and jump-preserving estimation, jump regression provides a natural statistical framework for image analysis. It enables statistical inference involved in many image processing problems including edge detection, image denoising and image deblurring. As the jump regression literature grows over the past two decades, the methodological development has far outpaced the availability of jump regression software. Although there are many image processing software programs, most of them implement either generic image operations or statistical analysis of images of specific types. In this article, we introduce DRIP, an R package with a number of state-of-the-art jump regression methods for jump detection and surface reconstruction. Its open-source nature and user-friendly interface make it convenient to develop and evaluate jump regression methods, encouraging wider adoption and future research. Although the current version of DRIP supports analysis of monochrome images only, as the literature on multivariate jump regression is still lacking, our methods can be applied separately to each color channel when handling color images.

1 Introduction

Rapid development in image acquisition technology has made images widely available in engineering, scientific research and everyday life. Because of their ability to capture spatial patterns and geometric information, images play increasingly important roles in areas such as astronomy, medical imaging, microscopy, motion tracking, optics, photography, and remote sensing, among others. Since image data are unstructured in nature and may contain measurement errors, analyzing images requires special techniques. In remote sensing, for instance, satellite images, which are often used in landscape monitoring, are affected by atmospheric noise due to light reflection from aerosols and clouds (Boonprong et al., 2018). In such cases, noise removal, i.e., image denoising (Buades et al., 2005; Polzehl and Spokoiny, 2000), may be necessary before any subsequent analysis. In medical diagnostics, magnetic resonance imaging (MRI) is used for measuring tumor growth in cancer patients. Images taken at different times may be misaligned because of the difference in the relative position between the patient and the imaging device. Meaningful comparison of the images for quantifying changes in tumor size cannot be done before geometrically matching up the images (Klein et al., 2009), a technique known as image registration (Zitova and Flusser, 2003). In photography, camera shake causes motion blur in images (Bai et al., 2018), making tasks such as recognizing objects in the image difficult. In such situations, a number of image deblurring techniques can be useful, e.g., Bayesian-based deblurring (Wang et al., 2011), diffusion-based deblurring (Yue et al., 2025) and deep learning-based deblurring (Quan et al., 2024).

There are a number of choices of software that implement various image processing techniques. MATLAB (The MathWorks Inc, 2023) includes an image processing toolbox with many useful tools for general-purpose image processing. A nice feature of MATLAB is that it has a built-in parallel computing toolbox to offer high-speed processing. In recent years, it has been expanded to support graphics processing unit (GPU) acceleration for some of its image processing functions (Georgantzoglou et al., 2014). A major limitation of MATLAB is that its commercial licensing can limit its large-scale adoption by practitioners. In particular, the licensing cost increases significantly for parallel computing, with per-worker pricing. Additionally, MATLAB's closed-source nature prevents users from modifying code for prototyping new methods, a common undertaking in scientific research. Among the open-source options, **OpenCV** (Bradski and Kaehler, 2008) is a comprehensive library for computer vision. It is traditionally written in C with most new algorithms developed in C++. As a result, **OpenCV** offers fast execution. Because of the same reason, it is more difficult for users to modify its code. More recently, **OpenCV** has added a Python interface to attract wider adoption. As an alternative to **OpenCV**, **scikit-image** (Van der Walt et al., 2014), written completely in Python, is an open-source library that provides users with easy access to algorithms for image processing. It offers a wide array of generic image operations such as image cropping, image resizing, and conversion between color and grayscale images. With the rising popularity of Python, **scikit-image** has seen significant growth in both adoption and contribution. Although favored by computer scientists, Python remains less popular than R (R Core Team, 2023) in the statistics community who are accustomed to R's rich ecosystem of complex statistical models. A number of R packages for image processing are available. Some of them are for general image processing. For example, the **EBImage** package

(Pau et al., 2010) provides general functionality for the reading, writing, processing and analysis of images. The **adimpro** package (Polzehl and Tabelow, 2007) and **aws** package (Polzehl and Tabelow, 2023; Polzehl et al., 2020) implement tools for manipulation of digital images and adaptive image denoising. The **OpenImageR** package (Mouselimis, 2023) offers functions for image filtering too. Its recent update allows the **Rcpp** functions of the **OpenImageR** package to be called in the C++ files of another package. Some other packages focus on images of certain types. For example, the **dti** package described in Tabelow et al. (2008) and Tabelow and Polzehl (2011) handles diffusion weighted imaging data with methods for structural adaptive smoothing and fiber tracking for tensor and tensor mixture models. The **landsat** package suggested in Goslee (2023) and Goslee (2011) provides radiometric and topographic correction routines for satellite imagery.

These existing image processing software either implement generic operations or carry out special-purpose analysis of images in specific categories. None of them has implemented methods in jump regression (Qiu, 2005), a general regression framework for image analysis. A major benefit with jump regression methods is that it enables statistical inference, such as surface estimation, standard error calculation and data-driven parameter selection, in many image processing problems including edge detection (Sun and Qiu, 2007), image denoising (Gijbels et al., 2006) and image deblurring (Hall and Qiu, 2007). The R package **DRIP** is created to fill this gap. Specifically, it implements two-dimensional jump regression methods with an emphasis on image processing applications. Regarding its computational performance, some of **DRIP**'s source code, large loops in particular, are written in Fortran or C to offer high execution speed. Parallel computing is also enabled in several routines to gain further acceleration. On the other hand, with the R interface, functions in **DRIP** are convenient to use, and their outputs are easy to visualize. As for its limitations, **DRIP** currently only supports analysis of grayscale images. This is because the existing jump regression literature has only considered cases of univariate responses. Much research is still needed on multivariate jump regression. As a practical workaround, our methods can be applied separately to each color channel when handling color images.

2 Package overview

The **DRIP** package provides functionality for three categories of image analysis problems: edge detection, edge-preserving noise removal and blind image deblurring. Edge detection is often used in image data compression. The following functions in the package are related to the task of edge detection: `stepEdge`, `roofEdge`, `dkQ`, `modify1` and `modify2`. Noise removal and image deblurring are important for improving human and machine perception of the images. The `JPLLK_surface`, `surfaceCluster` and `threeStage` functions in the package are related to noise removal. The `jpex` and `cv.jpex` functions are related to image deblurring. Furthermore, the package is designed for general image analysis without restrictions on the specific image type. However, the current use of **DRIP** focuses on grayscale images only, due to the aforementioned limitation in the jump regression literature. To use **DRIP** in practice, the workflow often involves parameter tuning before model estimation. This intended workflow is illustrated in the section of Workflow Illustration.

The remaining part of this article is organized as follows. Section 3 gives a short primer on 1-dimensional jump regression. Section 4 introduces 2-dimensional jump regression models, the related terminologies and their connections with image processing techniques. Section 5 describes edge detection routines in **DRIP**. Section 6 discusses methods included in the package for edge-preserving surface reconstruction from noisy images. Section 7 describes the problem of image deblurring and a blind deblurring algorithm in **DRIP**. Section 8 illustrates the workflow of using **DRIP**. Section 9 concludes the article and suggests future work.

3 A jump regression primer

The conventional 1-dimensional regression model has the following formulation:

$$Y_i = f(x_i) + \varepsilon_i, \quad i = 1, 2, \dots, n,$$

where $\{(x_i, Y_i)\}$ are observed data, and ε_i 's are random errors with mean 0 and variance σ^2 . This regression model implies that, at a given x position, Y is a random variable. Its mean value is a function of x . A major objective in regression analysis is to estimate the regression function $f(\cdot)$. A common assumption made by the conventional nonparametric regression methods (e.g., kernel smoothing) is that f is continuous. Then a reasonable approach to estimating f at position x is a weighted average

of Y_i 's whose x_i is near x . That is,

$$\hat{f}(x) = \frac{\sum_i Y_i K((x_i - x)/h)}{\sum_i K((x_i - x)/h)},$$

where $h > 0$ is a bandwidth parameter, and $K(\cdot)$ is a kernel function with support $[0, 1]$. The kernel function is often chosen such that observations closer to x receive heavier weights than those farther away. It can be proved that $\hat{f}(x)$ is a consistent estimator of $f(x)$, provided that f is a continuous function.

In the case where f is piece-wise continuous (i.e., it has jumps), the conventional kernel smoothing estimator $\hat{f}(x)$ is no longer consistent if x is a jump position. In fact, it converges to a value between $f_-(x)$ and $f_+(x)$, where $f_{\pm}(x)$ denotes the one-sided limits of f at x . Jump regression is designed to handle such scenarios and give rise to jump-preserving estimation. A fundamental idea in jump regression is to identify (explicitly or implicitly) the jump locations by estimating the one-sided limits $f_{\pm}$. For instance, position x would be considered a continuity point if the estimated difference between the two one-sided limits, $|\hat{f}_+(x) - \hat{f}_-(x)|$, is smaller than a certain threshold. Then $f(x)$ can be estimated as done in the conventional kernel smoothing. On the other hand, if $|\hat{f}_+(x) - \hat{f}_-(x)|$ is large, then x would be deemed a jump position, one of the one-sided estimates $\hat{f}_{\pm}(x)$ should be chosen for estimating $f(x)$.

One significant application of jump regression is image processing where the regression function $f(\cdot, \cdot)$ is 2-dimensional. In such cases, the jump positions form jump location curves, and there are infinitely many directions for defining one-sided limits of f . Consequently, it becomes more complex to construct one-sided estimates in 2-dimensional jump regression. However, the fundamental idea of jump-preserving estimation remains the same: estimate the regression surface by one-sided smoothing at jump points and conventional smoothing at continuity points. The next section discusses in more detail the connection between 2-dimensional jump regression and digital image representation.

4 Image representation and jump regression models

Images generated by various devices such as cameras, scanners and sensors capture the spatial and amplitude behavior of the related physical scene. The spatial location is described by the x - and y -coordinates. The amplitude behavior is described by the amounts of illumination. The amplitude of color images consists of three channels: red, green and blue. For grayscale images, the amplitude is only a scalar, also known as intensity or gray level. In our discussion, all images are grayscale unless mentioned otherwise.

Images can be continuous with respect to the spatial coordinates and amplitude. In order for computers to handle an image, the image needs to be digitized. The process of digitizing the spatial location of an image is referred to as *sampling* (Gonzalez and Woods, 2018). It takes equally spaced x - and y - coordinates and represents the image by a matrix $\{f(x_i, y_j) : i = 1, \dots, n_1, j = 1, \dots, n_2\}$. Each $f(x_i, y_j)$, the amplitude value at location (x_i, y_j) , is called a pixel of the image. The sample size $n_1 \times n_2$ is the image resolution. It is noteworthy that popular image processing software programs (e.g., MATLAB) have the origin of the spatial coordinates at the upper-left corner of the image with the x -axis going vertically downward and the y -axis going horizontally to the right. In R and thus with **DRIP**, however, the spatial coordinates of an image have their origin at the lower-left corner of the image, with the x -axis going horizontally to the right and the y -axis going vertically upward. In addition to digitizing the spatial location, image digitization also requires converting the intensity values into discrete quantities, a process often referred to as *quantization* (Qiu, 2005). Conventionally, $f(x_i, y_j)$ is discretized to be an integer in the range $[0, M - 1]$, where M is an integer power of 2. For instance, if $M = 2^8$, then a single pixel takes one byte in computer storage. Clearly, the quality of a digital image is affected by the sampling resolution and number of levels (i.e., M) used in quantization.

Observed digital images often contain noise which could be incurred from different sources. For example, noise can come from the digitization process described above. Noise can also be produced in poor lighting situations where images are taken. Assuming that the noise is additive to its true intensity values, an image can be described by the following two-dimensional regression model:

$$Z_{i,j} = f(x_i, y_j) + \varepsilon_{i,j}, \quad (x_i, y_j) \in \Omega, \quad i = 1, \dots, n_1, \quad j = 1, \dots, n_2, \quad (1)$$

where f is the true image intensity function, $\varepsilon_{i,j}$ denotes the noise at the (i, j) -th pixel, $Z_{i,j}$ is the observed intensity value at the (i, j) -th pixel, and Ω is the design space. The image intensity function f often has discontinuities. For instance, the boundary curves of objects in a photograph are positions where f has jumps. Because our human-eye systems have evolved to make use of the boundary curves for recognizing objects, jumps in f are important features of the image. This is illustrated in Figure

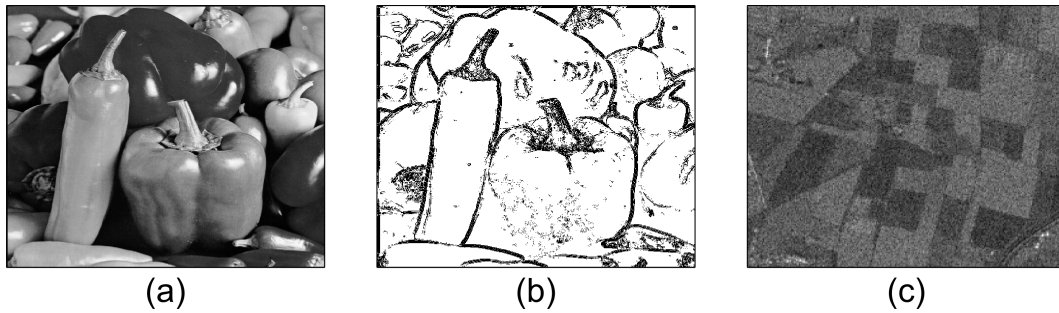


Figure 1: (a) The test image of peppers. (b) The boundary curves in the peppers image. (c) A SAR image of an area near Thetford forest, England.

1(a) – (b), which show the Lena image, a widely used test image in the image processing literature, and its jump positions (i.e., the boundary curves). Here two functions (`stepEdge` and `modify2`) have been used for extracting the jump positions. More details about these two functions and the topic of jump detection are provided in Section 4. It can be seen that much of the information regarding recognizing objects can be conveyed using jump locations only. Therefore, model (1) needs to be capable of handling discontinuous regression functions. The noise component is also necessary in (1). As an example, Figure 1(c) shows a synthetic aperture radar (SAR) image of an area near Thetford forest, England (Glasbey and Horgan, 1995). It can be seen that the SAR image is quite noisy. Both the Lena and SAR images are included in **DRIP** as test images. They can be accessed by the following code.

```
library(DRIP)
data("peppers")
data("sar")
```

The regression analysis in model (1) where the regression function is assumed to have jumps is referred to as *jump regression analysis* (Qiu, 2005). A major task in jump regression analysis is to estimate the jump locations. In some applications, it is also important to restore the true image from its observed but noisy version for better human and machine perception. Therefore, jump-preserving surface estimation is another important task in jump regression. The jump-preserving aspect is crucial due to the importance of jump locations. Formally, we assume that there exists a partition of the design space: $\Omega = \cup_{l=1}^L \Omega_l$ and $\Omega_{l_1} \cap \Omega_{l_2} = \emptyset$ if $l_1 \neq l_2$, and f is continuous in $\Omega_l \setminus \partial\Omega_l$ for $l = 1, \dots, L$. The point set $S = \cup_{l=1}^L \partial\Omega_l \cap \Omega^\circ$ are called the *jump location curves* of f and $\cup_{l=1}^L \Omega_l^\circ$ are called the *continuity regions*.

In image processing, jump locations in f are called *step edges*, and jump locations in the first-order derivatives of f are called *roof edges*. Therefore, edge detection and edge-preserving image restoration in image processing are essentially the same tasks as jump detection and jump-preserving surface estimation in jump regression. However, jump regression is more general in two regards. First, image pixels are equally spaced, but jump regression model (1) does not require this regularity. The design points can be arbitrarily distributed and the design space Ω can be of any shape. This flexibility can be useful in some applications such as the estimation of equi-temperature surfaces in meteorology and oceanography (Kang et al., 2021). Second, intensity values of a digital image are discrete, and some image processing techniques make use of this discreteness, rendering them inappropriate for handling certain jump regression problems. By contrast, jump regression treats $Z_{i,j}$ in (1) as a numerical response and does not share this limitation. To facilitate our discussion in the remainder of this article, we assume that $n_1 = n_2$, $\Omega = [0, 1] \times [0, 1]$, and $\{\epsilon_{i,j}\}$ are i.i.d. random errors with mean 0 and variance σ^2 . In the following sections, we describe some jump regression methods and their implementations in **DRIP**.

5 Estimation of jump location curves

5.1 Step edge detection

First, we briefly describe the step edge detection method proposed in Kang and Qiu (2014). For any given point $(x, y) \in \Omega$, consider its circular neighborhood,

$$O(x, y; h) = \{(u, v) \in \Omega : (x - u)^2 + (y - v)^2 \leq h^2\},$$

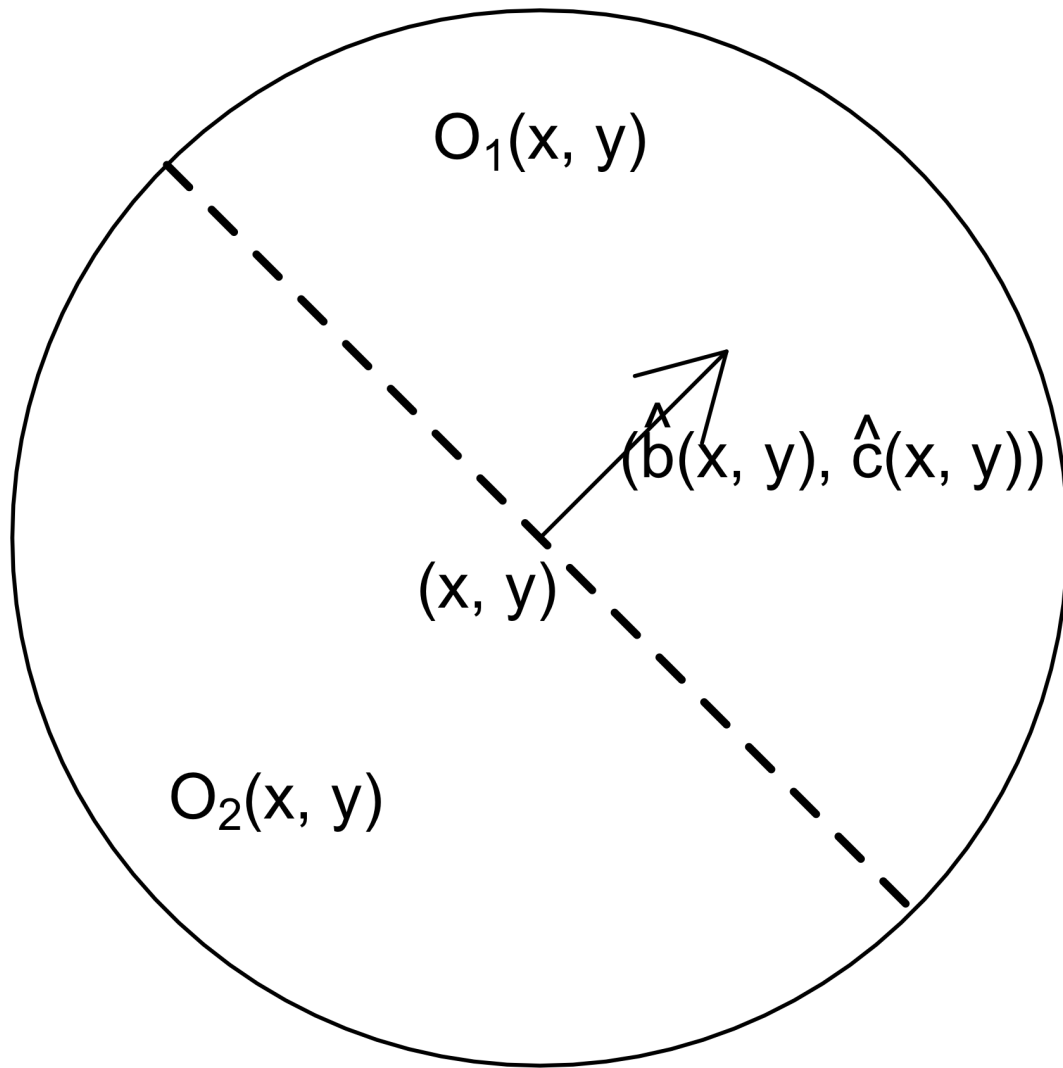


Figure 2: Neighborhood $O(x, y; h)$ of the design point (x, y) is divided into two halves $O_1(x, y)$ and $O_2(x, y)$ along the direction perpendicular to the estimated gradient $(\hat{b}(x, y), \hat{c}(x, y))$.

where $h > 0$ is a bandwidth parameter, and the following local linear kernel smoothing procedure:

$$\min_{a, b, c} \sum_{(x_i, y_j) \in O(x, y; h)} \left\{ Z_{i,j} - [a + b(x_i - x) + c(y_j - y)] \right\}^2 K_{i,j}, \quad (2)$$

where $K_{i,j} = K((x_i - x)/h, (y_j - y)/h)$ and K is a circularly symmetric bivariate density kernel function defined on the unit disk centered at the origin. The solution to (b, c) in (2), denoted by $(\hat{b}(x, y), \hat{c}(x, y))$, estimates the gradient of f at (x, y) . If (x, y) is near a jump location curve, $(\hat{b}(x, y), \hat{c}(x, y))$ indicates the direction in which f increases the fastest, i.e., orthogonal to the jump location curve's tangent. Based on this intuition, we divide $O(x, y; h)$ into two halves along the direction perpendicular to $(\hat{b}(x, y), \hat{c}(x, y))$. Specifically, define (as illustrated in Figure 2)

$$\begin{aligned} O_1(x, y) &= \left\{ (x_i, y_j) \in O(x, y; h) : (x_i - x)\hat{b}(x, y) + (y_j - y)\hat{c}(x, y) \geq 0 \right\}, \\ O_2(x, y) &= \left\{ (x_i, y_j) \in O(x, y; h) : (x_i - x)\hat{b}(x, y) + (y_j - y)\hat{c}(x, y) < 0 \right\}. \end{aligned}$$

The change in the values of f from $O_2(x, y)$ to $O_1(x, y)$ should be relatively large if (x, y) is a jump point. To estimate this change, we fit two local linear kernel smoothing procedures in $O_1(x, y)$ and

$O_2(x, y)$:

$$\min_{a_1, b_1, c_1} \sum_{(x_i, y_j) \in O_1(x, y)} \left\{ Z_{i,j} - [a_1 + b_1(x_i - x) + c_1(y_j - y)] \right\}^2 K_{i,j}, \quad (3)$$

$$\min_{a_2, b_2, c_2} \sum_{(x_i, y_j) \in O_2(x, y)} \left\{ Z_{i,j} - [a_2 + b_2(x_i - x) + c_2(y_j - y)] \right\}^2 K_{i,j}. \quad (4)$$

The solutions to a_1 of (3) and a_2 of (4) are denoted as $\hat{f}_+(x, y)$ and $\hat{f}_-(x, y)$, respectively. The design point (x, y) is flagged as a detected jump point if

$$|\hat{f}_+(x, y) - \hat{f}_-(x, y)| > u, \quad (5)$$

where u is a threshold parameter. This step edge detector is implemented by the `stepEdge` function, illustrated as follows.

```
stepedge <- stepEdge(image = sar, bandwidth = 10, thresh = 17,
                     degree = 1)
```

The output of `stepEdge` is a matrix of 0's and 1's. Figure 3(a) shows the estimated step edges by the function `stepEdge`. Here the parameter values of the bandwidth and threshold are user-specified. The data-driven parameter selection is discussed later in this section. It is worth noting that the bandwidth is specified in terms of the number of pixels. Also, we have exchanged 0's and 1's in the visualization with the `image` function in order to have the jump points shown in black.

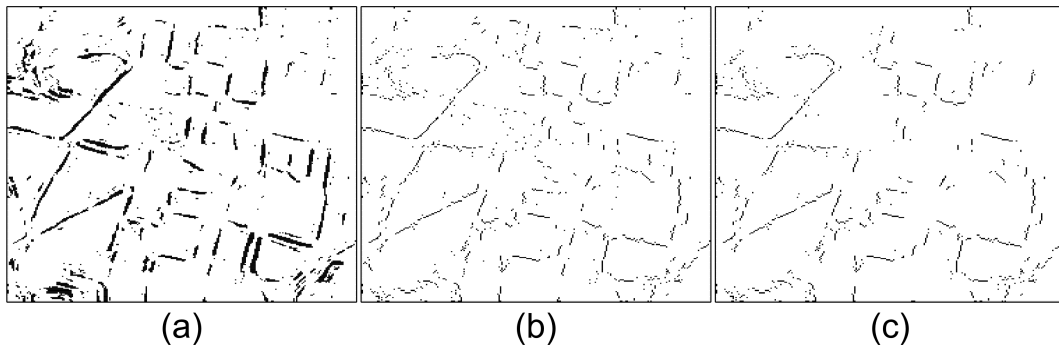


Figure 3: (a) The estimated step edges of SAR produced by `stepEdge`. (b) The estimated step edges after the first modification procedure described in the subsection of Two Modification Procedures. (c) The estimated step edges after the two modification procedures.

Next, we compare our step edge detector `stepEdge` with the classic Sobel edge detector implemented by the function `edge_detection` in the [OpenImageR](#) package. The Sobel edge detector estimates the partial derivatives using two 3×3 Sobel masks and outputs the magnitude of the estimated gradient. In Figure 4, we apply both edge detectors to a test image of peppers and its noisy version. The noise is generated from the normal distribution $N(0, 30^2)$. It can be seen from the figure that the two edge detectors have rendered comparable results for the original test image, but the Sobel edge detector has produced many scattered false signals for the noisy version. This is because the Sobel edge detector has masks of fixed size. By contrast, `stepEdge` can adjust the bandwidth value (i.e., the neighborhood size) to accommodate different levels of noise. Additionally, the Sobel edge detector does not generate the binary output of edge detection. We have used the 90% quantile as the threshold. In comparison, [DRIP](#) offers data-driven selection of the bandwidth and threshold parameters in `stepEdge`, which is described next.

```
edge_peppers <- stepEdge(image = peppers, bandwidth = 5, thresh = 14, degree = 1)
sobel_peppers <- edge_detection(image = peppers, method = "Sobel")
par(mfrow = c(2, 3), mar = c(0.1, 0.1, 0.1, 0.1), xaxt = "n", yaxt = "n")
image(peppers, col = gray(c(0:255)/255))
image(1 - edge_peppers, col = gray(c(0:255)/255))
image(sobel_peppers < quantile(sobel_peppers, 0.9), col = gray(c(0:255)/255))
set.seed(42)
noise <- matrix(rnorm(prod(dim(peppers))), mean = 0, sd = 30),
               nrow = nrow(peppers), ncol = ncol(peppers))
```

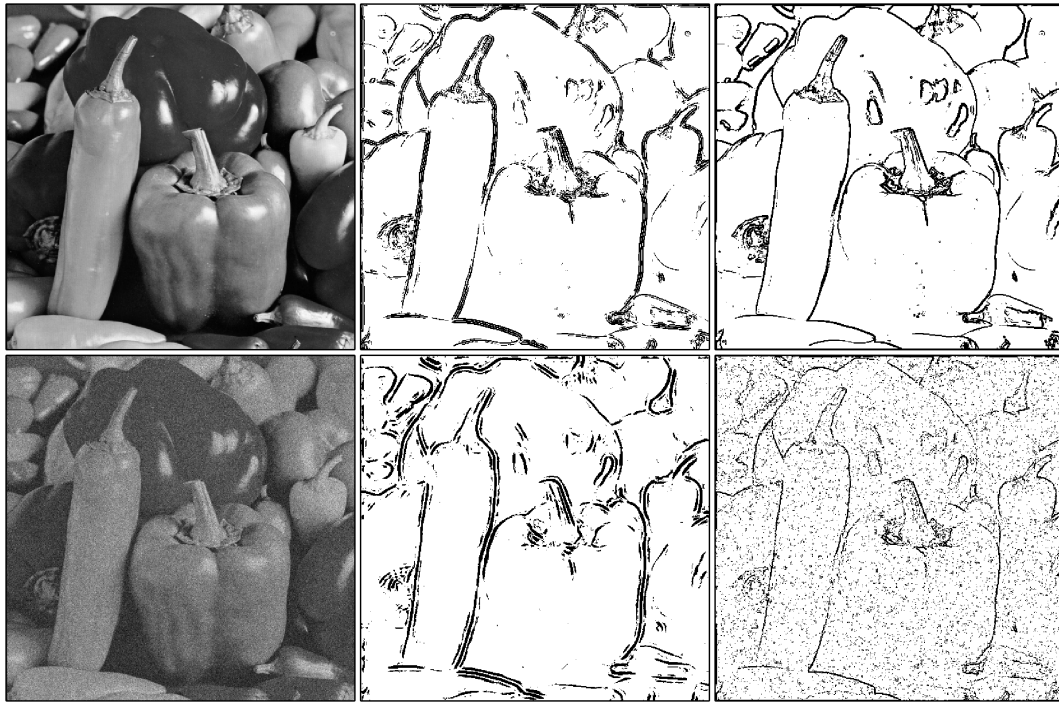


Figure 4: A comparison with the Sobel edge detector (from left to right). Top row: a test image of peppers, the edges detected by stepEdge and the edges detected by Sobel. Bottom row: a noisy version of the peppers image and the detected edges by stepEdge and Sobel.

```
edge_peppers <- stepEdge(image = peppers + noise, bandwidth = 11, thresh = 28,
                        degree = 1)
sobel_peppers <- edge_detection(image = peppers + noise, method = "Sobel")
image(peppers + noise, col = gray(c(0:255)/255))
image(1 - edge_peppers, col = gray(c(0:255)/255))
image(sobel_peppers < quantile(sobel_peppers, 0.9), col = gray(c(0:255)/255))
```

In order to properly select parameters for edge detection, [Kang and Qiu \(2014\)](#) has suggested the following performance measure for edge detectors.

$$d_{KQ}(\hat{S}, S; h, u) = \frac{1}{2|\hat{S}|} \sum_{(x', y') \in \hat{S}} d_E((x', y'), S) + \frac{1}{2|S|} \sum_{(x, y) \in S} d_E((x, y), \hat{S}), \quad (6)$$

where $d_E(\cdot, \cdot)$ denotes the Euclidean distance, and S and $\hat{S}$ are the true and estimated step edge points respectively. Here the Euclidean distance between a point (x', y') and a set S is defined to be $d_E((x', y'), S) = \inf_{(x, y) \in S} d_E((x', y'), (x, y))$. The first term in (6) includes all falsely detected jump points and calculates their average distance from S . The second term includes all missed true jump points and calculates their average distance from $\hat{S}$. If $\hat{S} = S$, we have $d_{KQ} = 0$. In practice, however, the true jump locations are unknown and thus d_{KQ} cannot be computed. To address this issue, a bootstrap version of (6), denoted by $\hat{d}_{KQ}(\hat{S}, S; h, u)$, is proposed in [Kang and Qiu \(2014\)](#). It works as follows.

Bootstrap Procedure for Selecting h and u

1. Apply the jump detector stepEdge to the original data set $\{(x_i, y_j, Z_{i,j}), i, j = 1, \dots, n\}$. The set of detected jump points is denoted as $\hat{S}$.
2. Apply a jump-preserving surface estimator and obtain $\hat{f}(x, y)$. Define the residuals by $\hat{\epsilon}_{i,j} = Z_{i,j} - \hat{f}(x_i, y_j)$.
3. Draw with replacement a bootstrap sample of size n^2 , denoted as $\{\hat{\epsilon}_{i,j}^*, i, j = 1, \dots, n\}$, from the residual set $\{\hat{\epsilon}_{i,j}, i, j = 1, \dots, n\}$, and then obtain a bootstrap data set $\{(x_i, y_j, Z_{i,j}^*), i, j = 1, \dots, n\}$, where $Z_{i,j}^* = \hat{f}(x_i, y_j) + \hat{\epsilon}_{i,j}^*$.

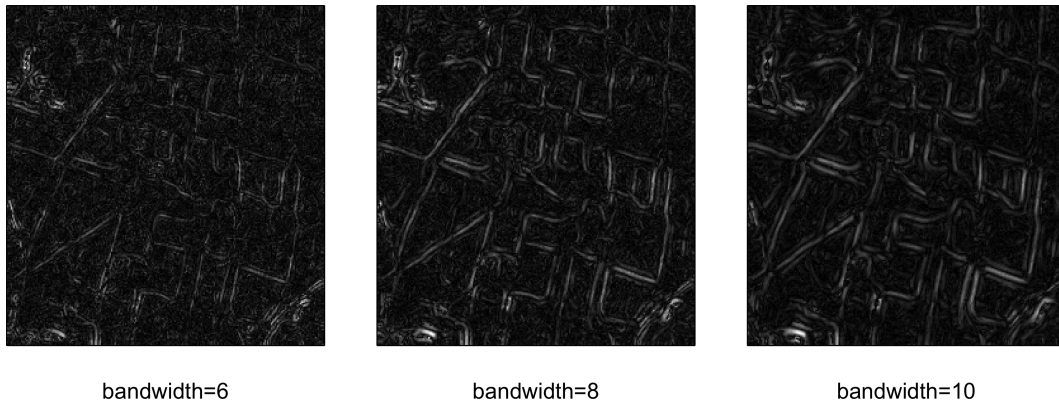


Figure 5: The images of the edge detection statistics for specified bandwidths.

4. Apply the jump detector with the same bandwidth and threshold as that in step 1 to the bootstrap data set, and obtain a set of detected jump points from this bootstrap data.
5. Repeat step 3 and 4 for B times. The sets of the detected jump points from the B bootstrap data sets are denoted as $\hat{S}_1^*, \hat{S}_2^*, \dots, \hat{S}_B^*$, respectively. Then, the bootstrap estimator of $d_{KQ}(\hat{S}, S; h, u)$ is defined by

$$\hat{d}_{KQ}(\hat{S}, S; h, u) = \frac{1}{B} \sum_{b=1}^B d_{KQ}(\hat{S}_b^*, \hat{S}; h, u).$$

The optimal bandwidth and threshold values are then approximated by the minimizer of $\hat{d}_{KQ}(\hat{S}, S; h, u)$ with respect to (h, u) :

$$\min_{h \in \mathcal{H}, u \in \mathcal{U}} \hat{d}_{KQ}(\hat{S}, S; h, u),$$

where $\mathcal{H}$ and $\mathcal{U}$ are candidates for the two parameters given by users.

In step 2 of the above procedure, $\hat{f}(x, y)$ can be constructed by any jump-preserving surface estimator proposed in the literature (Hillebrand and Müller, 2007; Qiu, 2007). In **DRIP**, we adopt the estimator suggested in Qiu (2009) (see Section 5 for more details about this surface estimator). It is worth noting that this bootstrap procedure is computationally intensive. The computational cost of applying the step edge detector once to an image of resolution $n \times n = N$ is $O(N^2)$. Hence the total computational cost of the bootstrap procedure is $O(|\mathcal{H}|BN^2)$, where $|\mathcal{H}|$ denotes the number of candidate bandwidths. **DRIP** offers a pilot selector `edgeParSelPilot()` that helps narrow the scope of the parameter search. With the pilot selector, users can specify several preliminary bandwidths. Since the bandwidth specifies the neighborhood size for local smoothing (3) – (4), positive integers in the range of $[1, 20]$ are usually reasonable candidates. The `edgeParSelPilot()` function would plot the image of the edge detection statistics (i.e., the left hand side of (5) at each pixel) for each given bandwidth. Users can narrow down the choices based on the visual impression. In addition, the function outputs a few upper quantiles of the edge detection statistics for each bandwidth. Those quantiles offer clues to what reasonable threshold values can be, as edges are usually sparse in an image. In the following illustration with the SAR image, we provide three bandwidths and three probabilities to `edgeParSelPilot()`.

```
edgeParSelPilot(sar, edgeType = "step", degree = 1,
               bandwidth = c(6, 8, 10), probs = c(0.75, 0.85, 0.95))
```

```
#>           probs=0.75 probs=0.85 probs=0.95
#> bandwidth=6   10.688440   13.79875   20.18085
#> bandwidth=8    9.610952   12.59395   19.38523
#> bandwidth=10   9.226387   12.21794   19.25910
```

Based on the visual impression given by Figure 5, bandwidth values around 10 seem reasonable as the visualization is less noisy. The quantiles suggest that the threshold should be around 20. Next, users can finalize the parameter choice using the function `stepEdgeParSel()`, which implements the above bootstrap procedure and estimates the edge detection performance for each combination of the bandwidth and threshold. Since this is a bootstrap procedure, the number of bootstrap samples needs to be given, and a random seed is required to ensure reproducibility. The function returns a matrix of

$\hat{d}_{KQ}$ values for each combination as well as the selected bandwidth and threshold parameters. The parameter selection in the following code chunk with the SAR image takes about 35 seconds to run on a single-core Intel(R) 4216 CPU @ 2.10GHz.

```
set.seed(24)
parSel <- stepEdgeParSel(image = sar, bandwidth = c(9, 10), degree = 1,
                        thresh = c(17, 21), nboot = 10)
print(parSel, type = "all")

#> The bootstrap matrix:
#>           thresh=17   thresh=21
#> bandwidth=9  0.009774721 0.012335871
#> bandwidth=10 0.008996036 0.011185702
#> The selected bandwidth: 10
#> The selected threshold: 17
```

DRIP has an alternative step edge detector, which is based on local constant kernel (LCK) smoothing. It is a more suitable choice if the intensity surface is piece-wise constant. The LCK-based edge detector can be called via `degree = 0`. The corresponding parameter selection can also be piloted by `edgeParSelPilot` and finalized by `stepEdgeParSel` with a `degree = 0` argument. In all these functions, the kernel K is chosen to be the bivariate Epanechnikov kernel $K(x, y) = (2/\pi)(1 - x^2 - y^2)I\{x^2 + y^2 \leq 1\}$ due to its nice theoretical properties (Fan and Gijbels, 2018).

5.2 Two modification procedures

There are two kinds of falsely flagged jump points. The first kind consists of those points near the real jumps. They occur due to the nature of local smoothing. This kind of deceptive jump points makes the detected edges thick. The second kind exists due to the nature of thresholding in (5). A design point can be flagged as a jump point with a small probability even if it is actually in a continuity region. Deceptive jump points of this kind are scattered in the entire design space. Qiu and Yandell (1997) has proposed two modification procedures to make the detected edges thin and to delete the scattered jump points. Their ideas are summarized below. Let $\theta_{i,j}$ denote the angle formed by the positive direction of the x-axis and the estimated gradient $(\hat{b}(x_i, y_j), \hat{c}(x_i, y_j))$ from solving (2) at (x_i, y_j) .

MP₁ (to make the detected edges thin). For each y_j , consider the design points on the line $y = y_j$, $\{(x_i, y_j) : i = 1, \dots, n\}$. Among them, the flagged jump points that satisfy $\theta_{i,j} \in [-\pi/4, \pi/4) \cup [3\pi/4, 5\pi/4)$ are denoted by $\{(x_{i_s}, y_j) : s = 1, \dots, m\}$. If there are integers $r_1 < r_2$ such that the increments of the sequence $x_{i_{r_1}} < x_{i_{r_1+1}} < \dots < x_{i_{r_2}}$ are all less than the neighborhood width $2h$, but $x_{i_{r_1}} - x_{i_{r_1-1}} > 2h$ and $x_{i_{r_2+1}} - x_{i_{r_2}} > 2h$, then $\{(x_{i_s}, y_j) : r_1 \leq s \leq r_2\}$ form a *tie*, and the middle point $((x_{i_{r_1}} + x_{i_{r_2}})/2, y_j)$ replaces the tie as the new jump point. Thus, each tie is reduced to one representative, its middle point. Along the y direction, for each x_i , consider the design points on the line $x = x_i$, $\{(x_i, y_j) : j = 1, \dots, n\}$. Perform the same modification as that along the x direction, except that only those jump points satisfying $\theta_{i,j} \in [\pi/4, 3\pi/4) \cup [5\pi/4, 7\pi/4)$ are considered.

When modifying the detected edges along the x direction, the condition $\theta_{i,j} \in [-\pi/4, \pi/4) \cup [3\pi/4, 5\pi/4)$ ensures that the possible edge curve forms an acute angle with the y -axis of less than or equal to $\pi/4$. This avoids canceling the true edges that are parallel with the x -axis.

MP₂ (to delete scattered jump points). For any jump point (x_i, y_j) , if the number of jump points in the neighborhood $O(x_i, y_j; h)$ is less than nh , then delete (x_i, y_j) from the set of jump points.

Functions `modify1` and `modify2` implement these two procedures. We apply them to modify the detected jumps of the SAR image.

```
steppedge_m1 <- edgeModify1(bandwidth = 10, image = sar, edge = steppedge,
                           plot = FALSE)
steppedge_m2 <- edgeModify2(bandwidth = 10, edge = steppedge_m1, plot = FALSE)
```

The modified versions are shown in Figure 3(b) – (c). It can be seen from the figure that the edges are thinner and there are fewer scattered jump points.

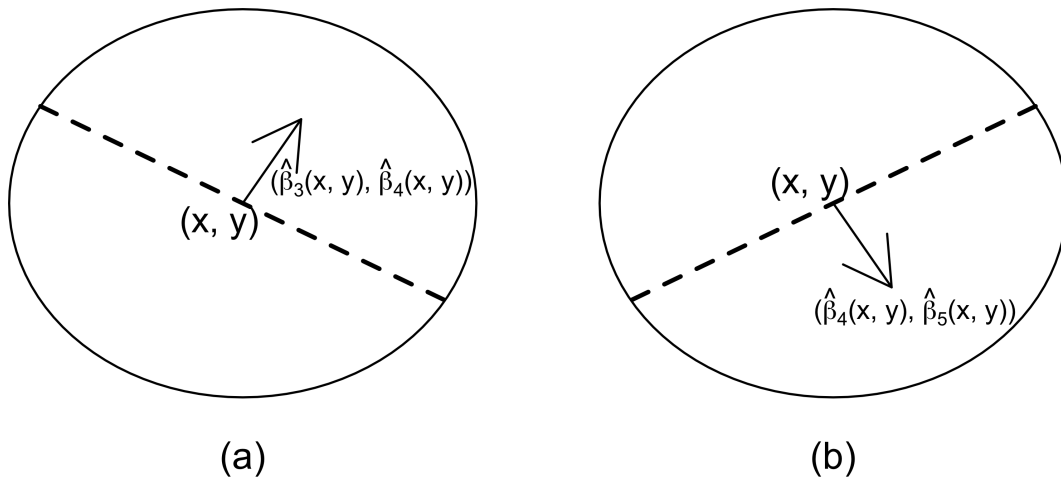


Figure 6: Neighborhood $O(x, y; h)$ of the design point (x, y) is divided into two halves along the direction (a) perpendicular to $(\hat{\beta}_3(x, y), \hat{\beta}_4(x, y))$, (b) perpendicular to $(\hat{\beta}_4(x, y), \hat{\beta}_5(x, y))$.

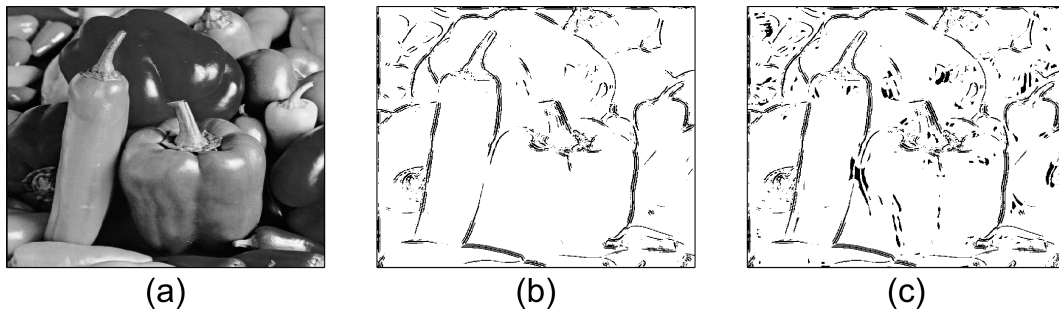


Figure 7: (a) A test image of peppers. (b) The detected step edges. (c) The detected step and roof edges.

5.3 Roof edge detection

Roof edge detection involves estimating the jump positions in the partial derivatives f'_x and f'_y . In [Qiu and Kang \(2015\)](#), the following local quadratic smoothing procedure is considered.

$$\min_{\beta \in \mathbb{R}^6} \sum_{(x_i, y_i) \in O(x, y; h)} \left\{ Z_{i,j} - \left[\beta_0 + \beta_1(x_i - x) + \beta_2(y_j - y) + \beta_3(x_i - x)^2 + \beta_4(x_i - x)(y_j - y) + \beta_5(y_j - y)^2 \right] \right\}^2 K_{i,j}, \quad (7)$$

where the bandwidth parameter h may be different from that in (2). The solutions to β_3 , β_4 and β_5 in (7) provide estimates for f''_{xx} , f''_{xy} and f''_{yy} . Similar to constructing $|\hat{f}_+ - \hat{f}_-|$ in the subsection of Step Edge Detection, we divide the neighborhood $O(x, y; h)$ into two halves along the direction perpendicular to $(\hat{\beta}_3(x, y), \hat{\beta}_4(x, y))$ and run two local linear kernel smoothing procedures separately in the two half circles to obtain the estimate $\hat{f}'_{x+} - \hat{f}'_{x-}$. This is illustrated in Figure 6(a). The estimation of $\hat{f}'_{y+} - \hat{f}'_{y-}$ is the same except that the neighborhood is divided along the direction perpendicular to $(\hat{\beta}_4(x, y), \hat{\beta}_5(x, y))$, as shown in Figure 6(b).

A pixel is flagged as a roof edge point if $M(x, y) = \max\{|\hat{f}'_{x+} - \hat{f}'_{x-}|, |\hat{f}'_{y+} - \hat{f}'_{y-}|\}$ is greater than a threshold value. However, this criterion can be large around step edges too. Therefore, roof edge detection should be done only at places where there are no step edges. In [DRIP](#), this roof edge detection method is implemented by `roofEdge`. We illustrate its use with a test image of peppers (Figure 7). The argument `edge1` specifies the step edge locations.

```
step.edges <- stepEdge(peppers, bandwidth = 6, thresh = 25)
roof.edges <- roofEdge(peppers, bandwidth = 9, thresh = 3000,
                      step_edge = step.edges)
```

Similar to the parameter selection for step edge detection, the bootstrap method works for roof edge detection. It is implemented by the `roofEdgeParSel` function, whose usage is identical to that

of `stepEdgeParSel`. The pilot selector `edgeParSelPilot` can be used via `edgeType = "roof"`. The two modification procedures can also be applied to enhance the visual appearance of the detected roof edges.

6 Jump-preserving surface estimation

6.1 A three-stage approach

In some applications, e.g., image denoising, the ultimate goal is to estimate the regression surface. Since jumps are important features that dominate human visual perception, they should be preserved by the estimation. Qiu (1998) has suggested a three-stage approach and its idea is outlined next. The first stage is to detect jumps and any reasonable jump detection procedure (e.g., the one implemented by `stepEdge`) can be used in this stage. In the second stage, at any given design point (x, y) , consider its neighborhood $O(x, y; h)$. Assume that the jump positions in $O(x, y; h)$ are denoted by $\{(w_r, v_r) : 1 \leq r \leq m\}$. Let (W, V) denote a random vector taking values over $\{(w_r, v_r)\}$. Let $\bar{w}$ and $\bar{v}$ be the sample means of $\{w_r\}$ and $\{v_r\}$, respectively. Then search for a line $a(W - \bar{w}) + b(V - \bar{v}) = 0$ such that it attains the following minimum.

$$\min_{\tilde{a}^2 + \tilde{b}^2 = 1} \text{Var} [\tilde{a}(W - \bar{w}) + \tilde{b}(V - \bar{v})].$$

The calculation of a and b is straightforward. We have

$$a = \sigma_{wv}, \quad b = \frac{1}{2} \left(\sigma_{vv} - \sigma_{ww} - \sqrt{(\sigma_{ww} - \sigma_{vv})^2 + 4\sigma_{wv}^2} \right),$$

where σ_{ww} , σ_{vv} and σ_{wv} denote the sample variances of $\{w_r\}$, $\{v_r\}$, and the sample covariance of $\{(w_r, v_r)\}$, respectively. Clearly, the line $a(W - \bar{w}) + b(V - \bar{v}) = 0$ is the first principal component line of the covariance matrix of (W, V) . It provides a first-order approximation to the jump location curve in $O(x, y; h)$. In the third stage, observations on the same side of the principal component line as (x, y) are combined in a Nadaraya-Watson kernel estimator.

$$\hat{f}(x, y) = \frac{\sum_{(x_i, y_i) \in \tilde{O}(x, y; h)} Z_{i,j} K_{i,j}}{\sum_{(x_i, y_i) \in \tilde{O}(x, y; h)} K_{i,j}}, \quad (8)$$

where $\tilde{O}(x, y; h)$ denotes the design points in $O(x, y; h)$ that are on the same side of the principal component line as (x, y) . That is, let $\zeta_{i,j} = a(x_i - \bar{w}) + b(y_i - \bar{v})$, and

$$\tilde{O}(x, y; h) = \begin{cases} \{(x_i, y_i) \in O(x, y; h) : \zeta_{i,j} \geq 0\}, & \text{if } a(x - \bar{w}) + b(y - \bar{v}) \geq 0; \\ \{(x_i, y_i) \in O(x, y; h) : \zeta_{i,j} < 0\}, & \text{if } a(x - \bar{w}) + b(y - \bar{v}) < 0. \end{cases}$$

If there are no detected jump points in $O(x, y; h)$, then $\tilde{O}(x, y; h)$ is $O(x, y; h)$. This three-stage approach is implemented by `threeStage`, as illustrated below. The detected step edges are supplied using the argument `edge1`. Notably, users can also input the detected roof edges using the argument `edge2`. In cases where $O(x, y; h)$ contains roof edge points, a principal component line is fitted to approximate the roof edge curve and (8) is defined similarly using the observations on the same side of the fitted principal component line. If $O(x, y; h)$ contains both step and roof edge points, a principal component line would be fitted using the step edge points only and roof edge points are ignored. This is because step edges dominate human visual perception in comparison with roof edges. The idea of incorporating roof edge features in (8) was proposed in Qiu and Kang (2015), as an improvement of the approach initially suggested by Qiu (1998). The resulting estimator preserves both step and roof edges of the image. In cases where roof edge detection is deemed unnecessary, users can simply provide a matrix of zeros to `edge2`. Figure 8(a) shows the estimated SAR image after applying `threeStage`. It can be seen that the estimation preserves the discontinuities at places where edges have been successfully detected. At places where the edge detector fails to flag the edge points, the estimation still blurs the jumps.

```
fit <- restore3Stage(image = sar, bandwidth = 4, step_edge = stepedge_m2,
  roof_edge = array(0, dim(sar)))
```

This three-stage approach requires a bandwidth parameter. It can be selected by minimizing the

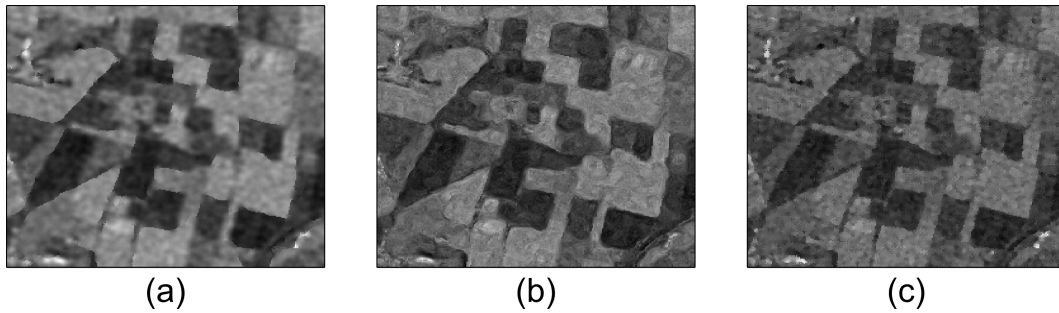


Figure 8: (a) The SAR image denoised by the three-stage approach. (b) The SAR image denoised by the gradient-based approach. (c) The SAR image denoised by the clustering-based approach.

following leave-one-out cross validation score.

$$\min_{h \in \mathcal{H}} \text{CV}(h) = \frac{1}{n^2} \sum_{i,j=1}^n \left[Z_{i,j} - \hat{f}_{-(i,j)}(x_i, y_j) \right]^2, \quad (9)$$

where $\mathcal{H}$ is a finite set of candidate bandwidths given by users and $\hat{f}_{-(i,j)}$ denotes the estimator in (8) without using the (i, j) -th pixel. The cross validation procedure selects the bandwidth that minimizes the prediction error, as observed image intensities are readily available. In edge detection, however, there is no observed version of edges and thus cross validation cannot be utilized. Bootstrap estimates are constructed instead. It is also worth noting that, similar to the bootstrap procedure, the leave-one-out cross validation procedure is computationally expensive, as it carries out the estimation $|\mathcal{H}|$ times. The `threeStageParSel` function implements the cross validation procedure.

```
bw_3stage <- restore3StageParSel(image = sar, bandwidth = 4:5,
                                step_edge = steledge_m2,
                                roof_edge = array(0, dim(sar)))

print(bw_3stage, type = "all")

#> The cross validation scores:
#>           bandwidth=4 bandwidth=5
#> CV-score    294.7589    303.8463
#> The selected bandwidth: 4
```

6.2 A gradient-based approach

Although the three-stage approach is intuitively appealing, it is still complicated to use, as it requires explicit edge detection which involves careful selection of two important parameters (bandwidth and threshold). Qiu (2009) suggested a direct surface estimation approach without using any procedure parameters other than the bandwidth. Here we briefly describe its idea. As in the subsection of Step Edge Detection, $O(x, y; h)$ is divided into $O_1(x, y)$ and $O_2(x, y)$ along the direction perpendicular to the estimated gradient. Consider the local linear kernel smoothing procedures (3) and (4) and denote their solutions by $\{(\hat{a}_k, \hat{b}_k, \hat{c}_k) : k = 1, 2\}$. If there are no jumps in $O(x, y; h)$, both $\hat{a}_1$ and $\hat{a}_2$ are consistent for estimating $f(x, y)$. If there is a single step edge in $O(x, y; h)$, either $O_1(x, y)$ or $O_2(x, y)$ should be mostly on one side of the edge, and the corresponding one-sided estimator should be able to estimate $f(x, y)$ well. In such cases, Qiu (2009) proposes to choose between $\hat{a}_1$ and $\hat{a}_2$ based on their weighted residual mean square of the fitted plane. Specifically, their proposed surface estimator is defined as

$$\hat{f}(x, y) = \begin{cases} \hat{a}_1, & \text{if } e_1(x, y) < e_2(x, y), \\ \hat{a}_2, & \text{if } e_1(x, y) > e_2(x, y), \\ (\hat{a}_1 + \hat{a}_2)/2, & \text{if } e_1(x, y) = e_2(x, y), \end{cases} \quad (10)$$

where $e_k(x, y) = \sum_{O_k(x, y)} \left\{ Z_{i,j} - [\hat{a}_k + \hat{b}_k(x_i - x) + \hat{c}_k(y_j - y)] \right\}^2 K_{i,j} / (\sum_{O_k(x, y)} K_{i,j})$ with $k = 1, 2$. The surface estimator in (10) can be obtained by function `JPLLK_surface`. The resulting image after applying it to the SAR image is shown in Figure 8(b). It is worth noting that a numeric vector of positive integers can be supplied to the bandwidth argument. The one with the smallest leave-one-out cross validation score, which is similarly defined as in (9), is chosen for fitting the regression surface. The function returns a list of the fitted surface, residuals, selected bandwidth and estimated noise level.

Here the residuals are defined as

$$\hat{\varepsilon}_{i,j} = Z_{i,j} - \hat{f}(x_i, y_j), \quad i, j = 1, \dots, n. \quad (11)$$

The residuals used in the bootstrap procedure in the subsection of Step Edge Detection are based on (11). The noise level σ is estimated by $\hat{\sigma} = (\sum_{i,j=1}^n \hat{\varepsilon}_{i,j}^2 / n^2)^{1/2}$.

```
fit_gb <- JPLLK_surface(image = sar, bandwidth = 3:7)
print(fit_gb, type = "bandwidth")
```

```
#> The selected bandwidth: 6
```

6.3 A clustering-based approach

The surface estimation approaches introduced in the previous two subsections work well at places where the jump location curve is smooth. This is because both approaches approximate the jump curve locally using a straight line. At places where the jump location curve has corners or cusps, such approximations may be inadequate. To address this limitation, Kang et al. (2018) proposed a clustering-based approach, described as follows. Consider the local linear kernel smoothing procedure (2). The resulting estimator, $\hat{a}(x, y)$, is good at removing noise but also blurs the edges. Center weighted median filtering (Ko and Lee, 1991; Sun et al., 1994) is a robust smoothing method in image processing and thus less sensitive to jumps in the regression surface. Apply the median filter with center weight $W_{0,0}$ to observations in $O(x, y; h)$ and denote its output by $\tilde{a}(x, y)$. Define

$$\Delta(x, y) = \hat{a}(x, y) - \tilde{a}(x, y).$$

If $O(x, y; h)$ is in a continuity region, then the regression surface in $O(x, y; h)$ should be approximated well by the fitted plane described by $(\hat{a}(x, y), \hat{b}(x, y), \hat{c}(x, y))$. As a result, $|\Delta(x, y)|$ should be small. If $O(x, y; h)$ contains a step edge, the fitted local plane cannot describe the regression surface in $O(x, y; h)$ well. Consequently, $|\Delta(x, y)|$ should be large. In other words, $|\Delta(x, y)|$ contains the jump information about f . Kang et al. (2018) has shown that, under the null hypothesis that f is continuous at (x, y) ,

$$nh [\hat{a}(x, y) - \tilde{a}(x, y)] \xrightarrow{d} N(0, \Sigma_{11} + \Sigma_{22} - 2\Sigma_{12}),$$

where

$$\Sigma_{11} = \sigma^2 \int \int_{\mathbb{R}^2} K(x, y)^2 dx dy, \quad \Sigma_{12} = \frac{E|\varepsilon_{1,1}|}{2\pi\phi(0)}, \quad \Sigma_{22} = \frac{1}{4\pi\phi(0)^2},$$

$\phi(\cdot)$ is the density function of $\varepsilon_{1,1}$, and $\xrightarrow{d}$ denotes convergence in distribution. Therefore, we flag (x, y) as a jump point if

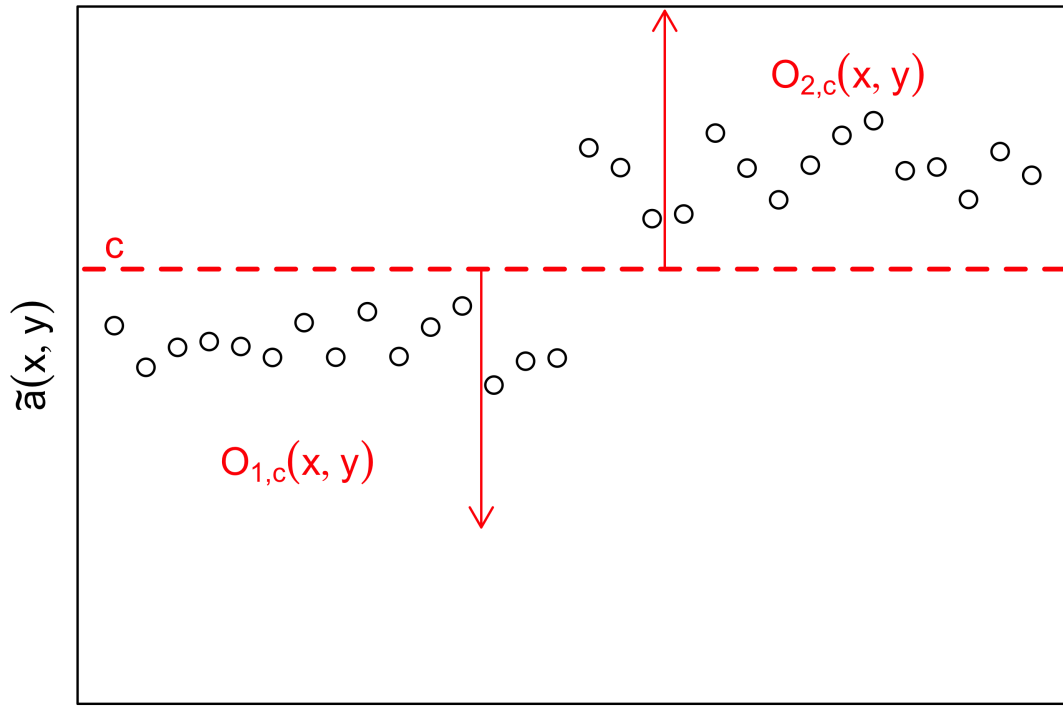
$$|\Delta(x, y)| > q_{1-\alpha} \sqrt{\frac{\Sigma_{11} + \Sigma_{22} - 2\Sigma_{12}}{nh}}, \quad (12)$$

where $q_{1-\alpha}$ denotes the $100(1 - \alpha)$ percentile of the standard normal distribution and $\alpha \in (0, 1)$ is the significance level. In cases where (12) holds, the estimation needs to accommodate the jump structure. Kang et al. (2018) has proposed to cluster the pixels in $O(x, y; h)$ into two groups based on their center weighted filtering outputs. Intuitively, pixels on the same side of the edge curve have similar filtering outputs. Hence, they can be put in the same group. Pixels on different sides of the edge curve have quite different filtering outputs, and they should be put in different groups. Clearly, it is not easy to specify the exact position of the edge curve within $O(x, y; h)$ and define the two groups of pixels accordingly. Nonetheless, an informative pixel clustering procedure can generate groups such that pixels within a group are similar in their filtering outputs and pixels in different groups have quite different filtering outputs. Such a pixel clustering procedure can reflect the local edge structure well without imposing restrictive conditions on the smoothness or shape of the edge curve. Specifically, define

$$\begin{aligned} O_{1,c}(x, y) &= \{(x_i, y_j) \in O(x, y; h) : \tilde{a}(x_i, y_j) \leq c\}, \\ O_{2,c}(x, y) &= O(x, y; h) \setminus O_{1,c}(x, y), \end{aligned}$$

where c is a cut-off constant and it is in the range

$$\left(\min_{(x_i, y_j) \in O(x, y; h)} \tilde{a}(x_i, y_j), \max_{(x_i, y_j) \in O(x, y; h)} \tilde{a}(x_i, y_j) \right).$$



A cross section of $O(x, y; h)$

Figure 9: An illustration of local pixel clustering.

The constant c is chosen to maximize the following clustering criterion

$$\frac{|O_{1,c}(x, y)| (\bar{v}_1 - \bar{v})^2 + |O_{2,c}(x, y)| (\bar{v}_2 - \bar{v})^2}{\sum_{(x_i, y_j) \in O_{1,c}(x, y)} (\tilde{a}(x_i, y_j) - \bar{v}_1)^2 + \sum_{(x_i, y_j) \in O_{2,c}(x, y)} (\tilde{a}(x_i, y_j) - \bar{v}_2)^2}, \quad (13)$$

where $\bar{v}_s$ denotes the sample mean of the center weighted filtering outputs within $O_{s,c}(x, y)$ for $s = 1, 2$, and $\bar{v}$ denotes the sample mean of the filtering outputs within $O(x, y; h)$. In (13), the numerator quantifies the dissimilarity between the two groups, and the denominator quantifies the dissimilarity within each of the two groups. Thus, it is reasonable to cluster the pixels by maximizing their ratio. It can be shown that (13) is the one-dimensional version of the clustering criterion suggested by [Friedman and Rubin \(1967\)](#). Notably, there are only finitely many cut-off constants that can result in different partitions. Thus, it is sufficient to evaluate (13) on the finite set of $\{\tilde{a}(x_i, y_j) : (x_i, y_j) \in O(x, y; h)\}$ and the maximization can be solved by exhaustive search. This is illustrated in Figure 9. It can be seen that the values of $\{\tilde{a}(x_i, y_j)\}$ indirectly indicate the local edge structure.

Next, a weighted average of observations in the same group as (x, y) should provide a good estimate for $f(x, y)$. Specifically,

$$\hat{f}(x, y) = \sum_{s=1}^2 \sum_{(x_i, y_j) \in O_{s,c}(x, y)} I\{(x, y) \in O_{s,c}(x, y)\} \frac{Z_{i,j} K_{i,j}}{\sum_{(x_i, y_j) \in O_{s,c}(x, y)} K_{i,j}}. \quad (14)$$

To use this estimator in practice, several unknown quantities in (12), σ^2 , $E|\varepsilon_{1,1}|$ and $\phi(0)$, need to be estimated. In the `surfaceCluster` function that implements this clustering-based approach, these unknown quantities are estimated using the residuals obtained from `JPLLK_surface`, as defined in (11). Specifically, the noise level $\hat{\sigma}$ is estimated the same way as the previous subsection. As for the other two,

$$\hat{\phi}(0) = \frac{1}{nb_n} \sum_{i,j=1}^n K_1\left(\frac{\hat{\varepsilon}_{i,j} - 0}{b_n}\right), \quad \widehat{E|\varepsilon_{1,1}|} = \frac{1}{n^2} \sum_{i,j=1}^n |\hat{\varepsilon}_{i,j}|,$$

where $K_1(\cdot)$ is the one-dimensional Gaussian kernel and $b_n = 1.06\hat{\sigma}n^{-2/5}$ ([Wand and Jones, 1994](#)). Next, we apply the clustering-based estimator (14) to the SAR image. The argument `cw` specifies the center weight used in the center weighted median filter. Its default value is 3. The percentile $q_{1-\alpha}$ is derived from the argument `sig.level` whose value specifies $1 - \alpha$. Since the SAR image is quite noisy,

setting a high significance level would blur many edges. We have used 0.7 in our illustration. The function returns a list of estimated image, $\hat{\sigma}$, $\hat{\phi}(0)$ and $\widehat{E|\varepsilon_{1,1}|}$. The estimated image is shown in Figure 8(c). It can be seen that some corners that have been blurred by the three-stage and clustering-based approaches are now preserved.

```
fit_cl <- surfaceCluster(image = sar, bandwidth = 3, sig.level = 0.7, cw = 3)
```

The bandwidth parameter in (14) can be chosen by minimizing the leave-one-out cross validation score, defined similarly as that in (9). The `surfaceCluster_bandwidth` function implements the bandwidth selection for this clustering-based estimator.

```
parSel_cluster <- surfaceCluster_bandwidth(image = sar, bandwidths = 3:4,
                                           sig.level = 0.7, cw = 3)
print(parSel_cluster, type = "all")
```

```
#> The cross validation scores:
#>   bandwidths      cv
#> 1          3 267.9241
#> 2          4 284.7558
#> The selected bandwidth: 3
#> The estimated sigma: 16.80083
#> The estimated value of the density at 0: 0.4199855
#> The estimated mean of absolute error: 0.776125
```

6.4 Comparison with other denoising methods

We compare with two other denoising algorithms available in **R**. The first is the adaptive weights smoothing (AWS) suggested by Polzehl and Spokoiny (2000). The AWS method is an iterative algorithm and known for its good performance in practical settings. The second is the non-local means (NL-means) method proposed by Buades et al. (2005). The NL-means filter has been shown to enjoy some nice asymptotic properties. Both AWS and NL-means have been implemented in the package `aws` (Polzehl and Tabelow, 2023; Polzehl et al., 2020).

We apply the `JPLLK_surface` function in **DRIP** (i.e., the gradient-based approach), and the `aws` and `nlmeans` functions in `aws` to the SAR image and the noisy peppers image shown in Figure 4. The denoised images are presented in Figure 10. It can be seen that the images restored by NL-means are still somewhat noisy. The AWS method has rendered the sharpest results, benefiting from the algorithm's adaptive design. This suggests that incorporating a certain adaptive mechanism into the jump regression approach can further improve its performance.

```
sarhat_aws <- aws(sar, hmax = 8)
fit_gb <- JPLLK_surface(image = sar, bandwidth = 3:7)
sarhat_nlm <- nlmeans(sar, lambda = 0.85, sigma = fit_gb$sigma, patchhw = 3,
                     searchhw = 3)
par(mfrow = c(2, 3), mar = c(0.1, 0.1, 0.1, 0.1), xaxt = "n", yaxt = "n")
image(fit_gb$fitted, col = gray(c(0:255)/255), zlim = range(sar))
image(sarhat_aws@theta, col = gray(c(0:255)/255), zlim = range(sar))
image(sarhat_nlm@theta, col = gray(c(0:255)/255), zlim = range(sar))
set.seed(42)
noise <- matrix(rnorm(prod(dim(peppers))), mean = 0, sd = 30),
               nrow = nrow(peppers), ncol = ncol(peppers))
peppershat_aws <- aws(peppers + noise, hmax = 8)
peppers_gb <- JPLLK_surface(image = peppers + noise, bandwidth = 3:7)
peppershat_nlm <- nlmeans(peppers + noise, lambda = 0.85, sigma = fit_gb$sigma,
                         patchhw = 3, searchhw = 3)
image(peppers_gb$fitted, col = gray(c(0:255)/255), zlim = range(peppers + noise))
image(peppershat_aws@theta, col = gray(c(0:255)/255),
      zlim = range(peppers + noise))
image(peppershat_nlm@theta, col = gray(c(0:255)/255),
      zlim = range(peppers + noise))
```



Figure 10: The denoised versions of the SAR image in Figure 1 and noisy peppers image in Figure 4 given by (from left to right) JPLLK_surface, aws and nlmeans, respectively.

7 Blind image deblurring

7.1 A deblurring model

In addition to having noise, images may also have blur involved. For instance, in astronomical imaging, ground-based imaging systems are subject to blurring due to the rapidly changing index of refraction of the atmosphere (Krist, 1995; Schulz, 1993). In photography, out-of-focus or camera shake often results in blurred images (Fergus et al., 2006). In medical imaging, blurred x-rays or mammograms are almost inevitable because the medical imaging systems limit the intensity of the incident radiation in order to protect the patient's health (Faulkner et al., 1989). In the image processing literature, a commonly used model to describe the relationship between the original image and its observed but blurred version is

$$Z(x_i, y_j) = G\{f\}(x_i, y_j) + \varepsilon_{i,j}, \quad (15)$$

where $G\{f\}$ is the convolution between g and f defined by

$$G\{f\}(x, y) = g \otimes f(x, y) = \int \int_{\Omega} g(x - u, y - v; x, y) f(u, v) \, du \, dv.$$

Here g is called the point spread function. It describes how the original image is spatially degraded (i.e., blurred). In most references, it is further assumed that the blurring is location (or spatially) invariant. That is, $g(u, v; x, y)$ does not depend on (x, y) . It can be seen that model (15) includes model (1) as a special case when the point spread function is a two-dimensional Dirac delta function with a point mass at the origin.

Blind image deblurring is for estimating f from Z when the point spread function g is not completely specified. This problem is ill-posed in nature because it is impossible to distinguish (g, f) from $(ag, a^{-1}f)$ based on the observed image Z alone, for any constant $a \neq 0$. Most existing blind image deblurring methods have been developed under the assumption that the point spread function g does not depend on location because the ill-posed nature of the deblurring problem would be exacerbated if the blur is spatially varying, making the problem significantly more challenging. In practice, however, this assumption is often invalid. In photography, for instance, realistic camera shake includes both translations and in-plane rotations (Levin et al., 2011). Such blurring cannot be accurately described by a location-invariant point spread function. Kang (2020) has suggested a deblurring procedure without assuming a location-invariant point spread function. Their approach, which is implemented in **DRIP**, is described in the next subsection.

7.2 Image deblurring using JPEX

In image processing, it is common to assume that the blurring preserves average brightness, i.e., the point spread function $g(\cdot, \cdot; x, y)$ is a bivariate density function for each (x, y) . To understand the ill-posed nature of the deblurring problem, consider a special case in the one-dimensional version of (15)

$$\eta = g_0 \otimes f_0(\xi),$$

where $g_0(\xi) = I(|\xi| \leq s)/(2s)$, $s > 0$ and $f_0(\xi) = I(\xi > 0)$. In this ideal case, the observations $\{(\xi_i, \eta_i) : i = 1, \dots, n\}$ contain no noise, there is a jump in f_0 , and the point spread function g_0 is corresponding to the one-dimensional out-of-focus blur with blurring extent s . By some straightforward calculation, we have

$$\eta(\xi) = \begin{cases} 0, & \text{if } \xi < -s, \\ \frac{1}{2s}(\xi + s), & \text{if } -s \leq \xi \leq s \\ 1, & \text{if } \xi > s. \end{cases}$$

Clearly, η is a piece-wise linear and continuous function. Without prior knowledge about s , it is impossible to distinguish the truth (g_0, f_0) from the no-blur solution (δ_0, η) based on the observations $\{(\xi_i, \eta_i) : i = 1, \dots, n\}$ alone, where $\delta_0(\cdot)$ denotes the Dirac delta function with the point mass at 0. It is also noteworthy that, because of its brightness-preserving property, the blurring does not alter $f_0(\xi)$ at places away from the jump location, where f_0 is flat. That is, $\eta(\xi) = f_0(\xi)$ for $\xi < -s$ or $\xi > s$. Therefore, the truth f_0 becomes estimable if we constrain the solution to the deblurring problem to be piece-wise constant, as non-zero gradients can only be caused by blurred jump structure in f_0 . Motivated by this observation, Kang (2020) has proposed to impose the piece-wise constant constraint when estimating f in (15). Specifically, it is assumed that f is constant on each design space partition Ω_l for $l = 1, \dots, L$. Let $\rho(x, y)$ denote the blurring extent at the pixel location (x, y) defined by

$$\rho(x, y) = \inf\{r > 0 : \text{supp } g(\cdot, \cdot; x, y) \subseteq O(x, y; r)\},$$

where supp denotes the function's support and $O(x, y; r) = \{(u, v) : (u - x)^2 + (v - y)^2 \leq r^2\}$. The method suggested by Kang (2020) assumes that $\rho(x, y) \equiv \rho$ but allows $g(u, v; x, y)$ to have different forms as (x, y) changes. Let $D = \{(x_i, y_j) \in \Omega, d_E((x_i, y_j), S) \leq \rho\}$. Obviously, D is a collection of pixels that are no more than one blurring extent away from the jump location curves. Thus, their observed intensities are altered by blur and these pixels are called *blurry* pixels. With the brightness-preserving property and the piece-wise constant constraint, pixels outside D (i.e., $(x_i, y_j) \in D^C$) are not affected by blur and they are called *sharp* pixels. The two kinds of pixels need to be handled differently. The key idea in Kang (2020) is to remove blur involved in blurry pixels using nearby sharp pixels.

For any given point $(x, y) \in \Omega$, consider its neighborhood $O(x, y; h)$, and fit the following local constant kernel smoothing and local linear kernel smoothing procedures:

$$\min_{\beta_0} \sum_{i,j=1}^n [Z_{ij} - \beta_0]^2 K_{ij}, \quad (16)$$

$$\min_{\beta_0, \beta_1, \beta_2} \sum_{i,j=1}^n [Z_{ij} - \beta_0 - \beta_1(x_i - x) - \beta_2(y_j - y)]^2 K_{ij}. \quad (17)$$

Let $\hat{\beta}_0^{(0)}(x, y)$, and $(\hat{\beta}_0^{(1)}(x, y), \hat{\beta}_1^{(1)}(x, y), \hat{\beta}_2^{(1)}(x, y))$ denote the solution to the minimization problem (16) and (17), respectively. Since $(\hat{\beta}_1^{(1)}(x, y), \hat{\beta}_2^{(1)}(x, y))$ estimates the gradient of $G\{f\}$ at (x, y) , its magnitude should be relatively small if (x, y) is a sharp pixel. Based on this intuition, the null hypothesis that

$$\mathbf{d}(G\{f\})(x, y) = (\partial_x G\{f\}(x, y), \partial_y G\{f\}(x, y)) = (0, 0) \quad (18)$$

can be tested using the following F statistic

$$F_L(x, y) = \frac{(\text{WRSS}_0(x, y) - \text{WRSS}_1(x, y)) / 2}{\tilde{\sigma}_n^2},$$

where $\tilde{\sigma}_n^2$ is a consistent estimate for σ^2 , $\text{WRSS}_0(x, y)$ and $\text{WRSS}_1(x, y)$ denote the weighted residual

sum of squares from (16) and (17), respectively. More specifically,

$$\begin{aligned} \text{WRSS}_0(x, y) &= \sum_{i,j=1}^n \left[Z_{ij} - \hat{\beta}_0^{(0)}(x, y) \right]^2 K_{ij}, \\ \text{WRSS}_1(x, y) &= \sum_{i,j=1}^n \left[Z_{ij} - \hat{\beta}_0^{(1)}(x, y) - \hat{\beta}_1^{(1)}(x, y)(x_i - x) - \hat{\beta}_2^{(1)}(x, y)(y_j - y) \right]^2 K_{ij}, \\ \hat{\sigma}_n^2 &= \frac{1}{n^2} \sum_{i,j=1}^n \left[Z_{ij} - \hat{\beta}_0^{(1)}(x_i, y_j) \right]^2. \end{aligned}$$

Here $\hat{\sigma}_n^2$ is different from $\hat{\sigma}^2$ in Section 4 and Section 5. It is directly based on the conventional local linear kernel smoothing, whereas $\hat{\sigma}^2$ is based on jump-preserving surface estimation. This is because the blurred regression surface $G\{f\}(x, y)$ is continuous and thus the residuals from (17) are appropriate representations of the random errors. This was not the case with jump regression problems in the previous two section.

Kang (2020) has shown that, under the null hypothesis (18), $2F_L(x, y)$ asymptotically follows a $\chi^2(2)$ distribution. Let $\chi_{1-\alpha}^2(2)$ denote the $100(1 - \alpha)$ percentile of $\chi^2(2)$, where $\alpha \in (0, 1)$ denotes the significance level. If $2F_L(x, y) > \chi_{1-\alpha}^2(2)$, then we flag (x, y) as a blurry pixel. Otherwise we conclude that f is constant on a neighborhood of (x, y) and the blur does not affect its observed intensity value. Then (x, y) is flagged as a sharp pixel. Let

$$\hat{D}_n = \left\{ (x_i, y_j) : 2F_L(x_i, y_j) > \chi_{1-\alpha}^2(2), i, j = 1, \dots, n \right\}. \quad (19)$$

The deblurring estimate for f is defined as follows.

$$\hat{f}(x, y) = \begin{cases} \hat{\beta}_0^{(0)}(x, y), & \text{if } 2F_L(x, y) \leq \chi_{1-\alpha}^2(2), \\ \hat{f}(\hat{x}^*, \hat{y}^*), & \text{otherwise,} \end{cases} \quad (20)$$

where $(\hat{x}^*, \hat{y}^*) \in \hat{D}_n^C$ such that $d_E((x, y), (\hat{x}^*, \hat{y}^*)) = d_E((x, y), \hat{D}_n^C)$. It can be seen that, in cases when (x, y) is flagged as a sharp pixel, the local constant kernel estimate in (16) is used for $\hat{f}(x, y)$. In cases when (x, y) is flagged as a blurry pixel, its own observed intensity was not used in the estimation. Instead, $\hat{f}(x, y)$ is a constant extrapolation from the closest detected sharp pixel. The idea is to make use of the sharp pixels in the same segment Ω_{l_0} as (x, y) for estimating $f(x, y)$, where l_0 is some integer in $1, \dots, L$ such that $(x, y) \in \Omega_{l_0}$. In order to obtain the correct extrapolation, it requires that (i) there exists some sharp pixels in Ω_{l_0} , and (ii) $(\hat{x}^*, \hat{y}^*) \in \Omega_{l_0}$ almost surely. If the first condition is violated, certain image segment is completely blurred with no sharp pixels in it and thus it cannot be estimated well by (20). The second condition is to ensure that the extrapolation is done within the correct image segment. This condition would hold if the blurring extent is constant.

The deblurring estimate (20) is essentially a jump-preserving extrapolation (JPEX) and is implemented by the `jpex` function. We apply it to a blurry stop-sign image, which is shown in Figure 11(a). The function returns a list of two: `deblurred`, the deblurred image, and `edge`, the detected blurry pixels. The deblurred stop-sign image and the detected blurry pixels are shown in Figure 11(b) and (c), respectively. It can be seen that the JPEX method is able to flag most blurry pixels (i.e., those in close vicinity of the object boundaries) and deblur the image well. The arguments `alpha` and `sigma` specify the significance level and noise level, respectively. The details about estimating the noise level in this stop-sign image are provided in the next subsection.

```
data("stopsign", package = "DRIP")
deblur <- jpex(image = stopsign, bandwidth = 2, sigma = 0.00623, alpha = 0.001)
names(deblur)
```

```
#> [1] "deblurred" "edge"
```

For comparison, in Figure 11(d) we also include the result given by the unsupervised Wiener-Hunt deconvolution function in `scikit-image`. It is a Bayesian method suggested by Orioux et al. (2010) that performs joint estimation of the point spread function and underlying image. It can be seen that the restored image still contains blur and has some ringing artifacts.

```
if (reticulate::py_module_available("skimage")) {
  skimage <- reticulate::import("skimage")
} else {
  message("Python module 'scikit-image' is required and being installed.")
}
```

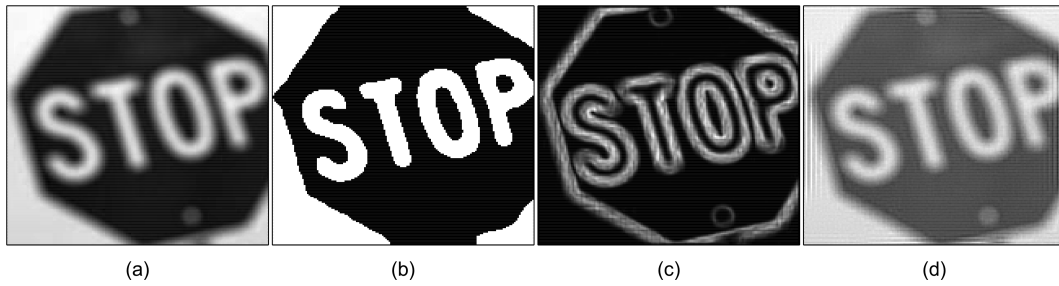


Figure 11: (a) The image of a blurry stop sign. (b) The deblurred image given by `jpex`. (c) The blurry pixels flagged by `jpex`. (d) The deblurred image given by `scikit-image`.

```
reticulate::py_install("scikit-image", pip = TRUE)
skimage <- reticulate::import("skimage")
}
np <- import("numpy")
restoration <- import("skimage.restoration")
psf0 <- np$ones(c(3L, 3L)) / 9
result <- restoration$unsupervised_wiener(stopsign, psf = psf0)
pyout <- result[[1]]
```

7.3 Bandwidth selection in JPEX

In the JPEX method, there is a bandwidth parameter h to choose. The parameter is used for two purposes: to flag possible blurry pixels in (19) and to estimate f at sharp pixel locations in (20). The purpose that h serves the JPEX method is similar to what the bandwidth parameter does in the conventional kernel smoothing. Thus, it is reasonable to select h by minimizing the leave-one-out cross validation score:

$$\min_h \sum_{i,j=1}^n \left[Z_{i,j} - \hat{\beta}_{0,-(i,j)}^{(1)}(x_i, y_j) \right]^2, \quad (21)$$

where $\hat{\beta}_{0,-(i,j)}^{(1)}$ denotes the estimate $\hat{\beta}_0^{(1)}$ from (17) when the (i, j) -th observation (x_i, y_j, Z_{ij}) is omitted. Since $\hat{\beta}_0^{(1)}$ is a linear combination of $Z_{i,j}$, (21) can be written as

$$\min_h \sum_{i,j=1}^n \left[\frac{Z_{i,j} - \hat{\beta}_0^{(1)}(x_i, y_j)}{1 - \omega} \right]^2, \quad (22)$$

where $\omega = K(0, 0)/r_{00}$ and $r_{00} = \sum_{i,j=1}^n K_{i,j}$. The computation in (22) is greatly simplified as the local linear kernel smoothing procedure (17) does not need to be carried out n^2 times. In the `DRIP` package, this is implemented by the `cv.jpex` function, as illustrated below. There is an option in `cv.jpex` for parallel computing. The argument `ncpus` gives the number of cores to use, i.e., at most how many child processes will be run simultaneously. The value must be at least one. The output of `cv.jpex` includes (i) LLK, the estimated surface given by $\hat{\beta}_0^{(1)}$ from (17), (ii) `sigma`, the estimated noise level $\hat{\sigma}_n$, defined in the previous subsection, (iii) `cv`, the cross validation scores corresponding to the bandwidth candidates, (iv) `bandwidths`, the input bandwidth candidates, and (v) `band.min`, the bandwidth with the smallest cross validation score.

```
cv.out <- cv.jpex(image = stopsign, bandwidths = c(2, 3), ncpus = 2)
print(cv.out, type = "all")
```

```
#> The cross validation scores:
#> [1] 5.591376e-05 1.421656e-04
#> The selected bandwidth: 2
#> The estimated sigma: 0.006231292
```

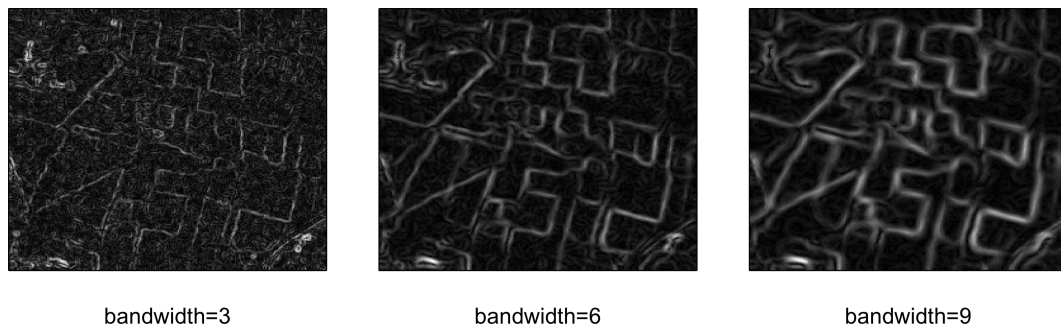


Figure 12: The images of the step edge detection statistics with the specified bandwidth values.

8 Workflow illustration

The workflow of using functions in **DRIP** often involves tuning some parameters first and then estimating the image features. Here we illustrate this process using edge-preserving noise removal of the SAR image as an example. Specifically, in the edge detection step, the edge detector `stepEdge` requires a bandwidth parameter and a threshold value. We first establish an initial range for the parameters. The visualization provided by the `edgeParSelPilot` function is useful for this purpose.

```
edgeParSelPilot(sar, edgeType = "step", degree = 0,
               bandwidth = c(3, 6, 9), probs = c(0.75, 0.8, 0.9, 0.95))

#>           probs=0.75 probs=0.8 probs=0.9 probs=0.95
#> bandwidth=3  15.82019  17.36624  22.11585  26.87138
#> bandwidth=6  13.31046  15.28433  21.32755  26.51575
#> bandwidth=9  14.72569  16.76905  22.61165  27.09358
```

It can be seen from Figure 12 that the bandwidth value of 3 seems too small (the corresponding image is still noisy). The bandwidth value of 9 appears to be too large (minor edges are lost in the resulting image). Therefore, bandwidth values around 6 are reasonable choices. Next, we determine the initial range for the threshold parameter. Based on the upper quantiles of the detection statistics with bandwidth 6, threshold values around 15 are good starting points. Next, we can finalize our parameter selection using the function `stepEdgeParSel`.

```
set.seed(24)
bandthresh <- stepEdgeParSel(sar, bandwidth = 4:9, thresh = 15:16,
                           nboot = 10, degree = 0)
edge1 <- stepEdge(sar, bandwidth = bandthresh$bandwidth,
                 thresh = bandthresh$threshold, degree = 0)
```

With the selected bandwidth and threshold, we have detected the step edges in the SAR image (shown in Figure 13). In the noise removal step, we apply the three-stage approach described earlier. Since the three-stage approach also requires a bandwidth parameter, we determine its value by cross validation (CV) before estimating the image. The denoised image is shown in Figure 13 and this completes the workflow.

```
cv3stage <- restore3StageParSel(sar, bandwidth = 4:9, step_edge = edge1,
                              roof_edge = array(0, dim(sar)))
sar_out <- restore3Stage(sar, bandwidth = cv3stage$bandwidth,
                       step_edge = edge1, roof_edge = array(0, dim(sar)))
par(mfrow = c(1, 2), mar = c(1, 1, 1, 1), xaxt = "n", yaxt = "n")
image(1 - edge1, col = gray(c(0:1)))
image(sar_out, col = gray(c(0:255)/255))
```

9 Concluding remarks

We have described **DRIP**, an R package that implements a number of jump regression methods with an emphasis on image processing applications. The package provides functions for step edge detection, roof edge detection, image denoising, image deblurring and the related data-driven parameter

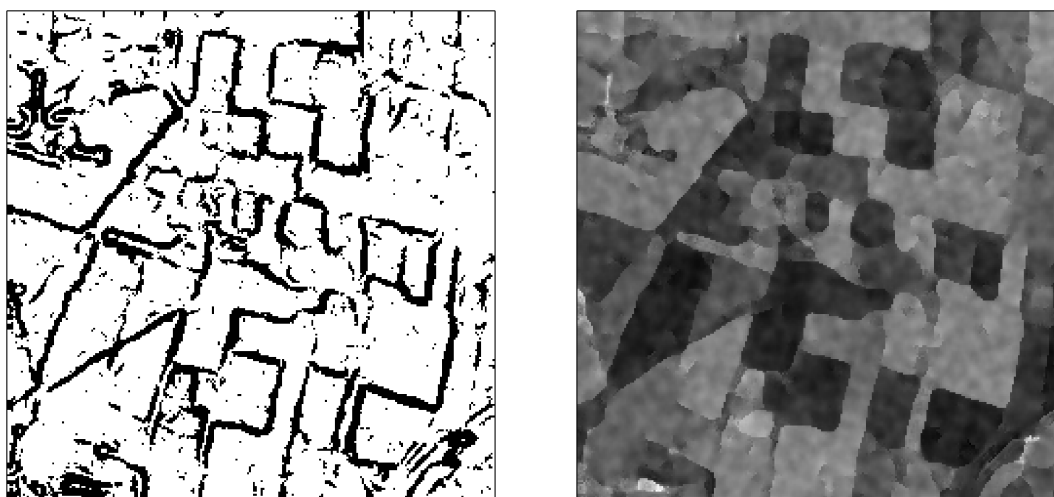


Figure 13: The detected step edges using the bootstrap-selected bandwidth and threshold values (left). The denoised SAR image using the CV-selected bandwidth (right).

selection. Much of **DRIP**'s source code is written in Fortran or C to achieve good computational performance. Its R interface enables easy adoption and adaptation of the methods. It is intended to further broaden the scope of the package in the future. In particular, jump regression-based image registration (Qiu and Xing, 2013) has been an active research area. Including those routines in the package is certainly an interesting direction to pursue. Adding methods for image comparison (Feng and Qiu, 2018) and image monitoring (Kang, 2022, 2023) is also planned for the future.

References

- Y. Bai, G. Cheung, X. Liu, and W. Gao. Graph-based blind image deblurring from a single photograph. *IEEE Transactions on Image Processing*, 28(3):1404–1418, 2018. doi: TIP.2018.2874290. [p71]
- S. Boonprong, C. Cao, W. Chen, X. Ni, M. Xu, and B. K. Acharya. The classification of noise-afflicted remotely sensed data using three machine-learning techniques: Effect of different levels and types of noise on accuracy. *International Journal of Geo-Information*, 7(7), 2018. doi: 10.3390/ijgi7070274. [p71]
- G. Bradski and A. Kaehler. *Learning OpenCV: Computer vision with the OpenCV library*. "O'Reilly Media, Inc.", 2008. URL <https://opencv.org/>. [p71]
- A. Buades, B. Coll, and J.-M. Morel. A review of image denoising algorithms, with a new one. *Multiscale modeling & simulation*, 4(2):490–530, 2005. doi: 10.1137/040616024. [p71, 85]
- J. Fan and I. Gijbels. *Local Polynomial Modelling and Its Applications: Monographs on Statistics and Applied Probability* 66. Routledge, New York, 2018. doi: 10.1201/9780203748725. [p79]
- K. Faulkner, C. Kotre, and M. Louka. Veiling glare deconvolution of images produced by X-ray image intensifiers. In *Third International Conference on Image Processing and its Applications*, pages 669–673. IET, 1989. URL <https://ieeexplore.ieee.org/abstract/document/132213>. [p86]
- L. Feng and P. Qiu. Difference detection between two images for image monitoring. *Technometrics*, 60(3):345–359, 2018. doi: 10.1080/00401706.2017.1356378. [p91]
- R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis, and W. T. Freeman. Removing camera shake from a single photograph. In *Acm Siggraph 2006 Papers*, pages 787–794. ACM, 2006. doi: 10.1145/1179352.1141956. [p86]
- H. P. Friedman and J. Rubin. On some invariant criteria for grouping data. *Journal of the American Statistical Association*, 62(320):1159–1178, 1967. doi: 10.2307/2283767. [p84]
- A. Georgantzoglou, J. da Silva, and R. Jena. Image processing with matlab and gpu. In *MATLAB Applications for the Practical Engineer*. IntechOpen, 2014. doi: 10.5772/58300. [p71]

- I. Gijbels, A. Lambert, and P. Qiu. Edge-preserving image denoising and estimation of discontinuous surfaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(7):1075–1087, 2006. doi: 10.1109/TPAMI.2006.140. [p72]
- C. A. Glasbey and G. W. Horgan. *Image Analysis for The Biological Sciences*. John Wiley & Sons, Hoboken, New Jersey, 1995. URL <https://dl.acm.org/doi/abs/10.5555/208252>. [p74]
- R. C. Gonzalez and R. E. Woods. *Digital Image Processing*. Pearson, New York, NY, 4 edition, 2018. URL <https://elibrary.pearson.de/book/99.150005/9781292223070>. [p73]
- S. Goslee. *landsat: Radiometric and Topographic Correction of Satellite Imagery*, 2023. URL <https://CRAN.R-project.org/package=landsat>. R package version 1.1.2. [p72]
- S. C. Goslee. Analyzing remote sensing data in R: the landsat package. *Journal of Statistical Software*, 43: 1–25, 2011. doi: 10.18637/jss.v043.i04. [p72]
- P. Hall and P. Qiu. Blind deconvolution and deblurring in image analysis. *Statistica Sinica*, pages 1483–1509, 2007. URL <http://www.jstor.org/stable/24307685>. [p72]
- M. Hillebrand and C. H. Müller. Outlier robust corner-preserving methods for reconstructing noisy images. *The Annals of Statistics*, 35(1):132 – 165, 2007. doi: 10.1214/009053606000001109. [p78]
- Y. Kang. Consistent blind image deblurring using jump-preserving extrapolation. *Journal of Computational and Graphical Statistics*, 29(2):372–382, 2020. doi: 10.1080/10618600.2019.1665536. [p86, 87, 88]
- Y. Kang. Statistical quality control using image intelligence: A sparse learning approach. *Naval Research Logistics (NRL)*, 69(7):996–1008, 2022. doi: 10.1002/nav.22069. [p91]
- Y. Kang. Statistical process monitoring approaches for image data. *Wiley StatsRef: Statistics Reference Online*, pages 1–15, 2023. doi: 10.1002/9781118445112.stat08263. [p91]
- Y. Kang and P. Qiu. Jump detection in blurred regression surfaces. *Technometrics*, 56(4):539–550, 2014. doi: 10.1080/00401706.2013.844732. [p74, 77]
- Y. Kang, P. S. Mukherjee, and P. Qiu. Efficient blind image deblurring using nonparametric regression and local pixel clustering. *Technometrics*, 60(4):522–531, 2018. doi: 10.1080/00401706.2017.1415975. [p83]
- Y. Kang, Y. Shi, Y. Jiao, W. Li, and D. Xiang. Fitting jump additive models. *Computational Statistics & Data Analysis*, 162:107266, 2021. doi: 10.1016/j.csda.2021.107266. [p74]
- A. Klein, J. Andersson, B. A. Ardekani, J. Ashburner, B. Avants, M.-C. Chiang, G. E. Christensen, D. L. Collins, J. Gee, P. Hellier, et al. Evaluation of 14 nonlinear deformation algorithms applied to human brain mri registration. *Neuroimage*, 46(3):786–802, 2009. doi: 10.1016/j.neuroimage.2008.12.037. [p71]
- S.-J. Ko and Y. H. Lee. Center weighted median filters and their applications to image enhancement. *IEEE Transactions on Circuits and Systems*, 38(9):984–993, 1991. doi: 10.1109/31.83870. [p83]
- J. Krist. Simulation of HST PSFs using tiny tim. In R. Shaw, H. Payne, and J. Hayes, editors, *Astronomical Data Analysis Software and Systems IV*, volume 77 of *ASP Conference Series*, pages 349 – 352, 1995. URL <https://adsabs.harvard.edu/full/record/seri/ASPC./0077/1995ASPC...77..349K.html>. [p86]
- A. Levin, Y. Weiss, F. Durand, and W. T. Freeman. Understanding blind deconvolution algorithms. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(12):2354–2367, 2011. doi: 10.1109/TPAMI.2011.148. [p86]
- L. Mouselimis. *OpenImageR: An Image Processing Toolkit*, 2023. URL <https://CRAN.R-project.org/package=OpenImageR>. R package version 1.3.0. [p72]
- F. Orieux, J.-F. Giovannelli, and T. Rodet. Bayesian estimation of regularization and point spread function parameters for wiener–hunt deconvolution. *Journal of the Optical Society of America A*, 27(7): 1593–1607, 2010. doi: 10.1364/JOSAA.27.001593. [p88]
- G. Pau, F. Fuchs, O. Sklyar, M. Boutros, and W. Huber. EBIImage - an R package for image processing with applications to cellular phenotypes. *Bioinformatics*, 26(7):979–981, 2010. doi: 10.1093/bioinformatics/btq046. [p72]

- J. Polzehl and V. G. Spokoiny. Adaptive weights smoothing with applications to image restoration. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 62(2):335–354, 2000. doi: 10.1111/1467-9868.00235. [p71, 85]
- J. Polzehl and K. Tabelow. Adaptive smoothing of digital images: The R package adimpro. *Journal of Statistical Software*, 19(1):1–17, 2007. doi: 10.18637/jss.v019.i01. [p72]
- J. Polzehl and K. Tabelow. *Magnetic Resonance Brain Imaging: Modeling and Data Analysis Using R*. UseR! Springer, 2023. doi: 10.1007/978-3-030-29184-6. [p72, 85]
- J. Polzehl, K. Papafitsoros, and K. Tabelow. Patch-wise adaptive weights smoothing in R. *Journal of Statistical Software*, 95(6):1–27, 2020. doi: 10.18637/jss.v095.i06. [p72, 85]
- P. Qiu. Discontinuous regression surfaces fitting. *The Annals of Statistics*, 26(6):2218–2245, 1998. doi: 10.1214/aos/1024691468. [p81]
- P. Qiu. *Image Processing and Jump Regression Analysis*. John Wiley & Sons, Hoboken, New Jersey, 2005. URL <https://www.wiley.com/en-be/Image+Processing+and+Jump+Regression+Analysis-p-9780471420996>. [p72, 73, 74]
- P. Qiu. Jump surface estimation, edge detection, and image restoration. *Journal of the American Statistical Association*, 102(478):745–756, 2007. doi: 10.1198/016214507000000301. [p78]
- P. Qiu. Jump-preserving surface reconstruction from noisy data. *Annals of the Institute of Statistical Mathematics*, 61:715–751, 2009. doi: 10.1007/s10463-007-0166-9. [p78, 82]
- P. Qiu and Y. Kang. Blind image deblurring using jump regression analysis. *Statistica Sinica*, 25: 879–899, 2015. doi: 10.5705/ss.2014.054. [p80, 81]
- P. Qiu and C. Xing. On nonparametric image registration. *Technometrics*, 55(2):174–188, 2013. doi: 10.1080/00401706.2012.727768. [p91]
- P. Qiu and B. Yandell. Jump detection in regression surfaces. *Journal of Computational and Graphical Statistics*, 6(3):332–354, 1997. doi: 10.2307/1390737. [p79]
- Y. Quan, Z. Wu, R. Xu, and H. Ji. Deep single image defocus deblurring via gaussian kernel mixture learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):11361–11377, 2024. doi: 10.1109/TPAMI.2024.3457856. [p71]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2023. URL <https://www.R-project.org/>. [p71]
- T. J. Schulz. Multiframe blind deconvolution of astronomical images. *Journal of the Optical Society of America A*, 10(5):1064–1073, 1993. doi: 10.1364/JOSAA.10.001064. [p86]
- J. Sun and P. Qiu. Jump detection in regression surfaces using both first-order and second-order derivatives. *Journal of Computational and Graphical Statistics*, 16(2):289–311, 2007. doi: 10.1198/106186007X204753. [p72]
- T. Sun, M. Gabbouj, and Y. Neuvo. Center weighted median filters: some properties and their applications in image processing. *Signal Processing*, 35(3):213–229, 1994. doi: 10.1016/0165-1684(94)90212-7. [p83]
- K. Tabelow and J. Polzehl. Beyond the gaussian model in diffusion-weighted imaging: The package dti. *Journal of Statistical Software*, 44(12):1–26, 2011. URL <https://www.jstatsoft.org/v44/i12/>. [p72]
- K. Tabelow, J. Polzehl, V. Spokoiny, and H. U. Voss. Diffusion tensor imaging: Structural adaptive smoothing. *NeuroImage*, 39(4):1763–1773, 2008. doi: 10.1016/j.neuroimage.2007.10.024. [p72]
- . The MathWorks Inc. *MATLAB version: 23.2.0 (R2023b)*. The MathWorks Inc., Natick, Massachusetts, United States, 2023. URL <https://www.mathworks.com>. [p71]
- S. Van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu. scikit-image: image processing in python. *PeerJ*, 2:e453, 2014. doi: 10.7717/peerj.453. [p71]
- M. P. Wand and M. C. Jones. *Kernel smoothing*. CRC Press, Boca Raton, FL, 1994. doi: 10.1201/b14876. [p84]

- C. Wang, L. Sun, P. Cui, J. Zhang, and S. Yang. Analyzing image deblurring through three paradigms. *IEEE Transactions on Image Processing*, 21(1):115–129, 2011. doi: 10.1109/TIP.2011.2159985. [p71]
- Z. Yue, J. Wang, and C. C. Loy. Efficient diffusion model for image restoration by residual shifting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(1):116–130, 2025. doi: 10.1109/TPAMI.2024.3461721. [p71]
- B. Zitova and J. Flusser. Image registration methods: a survey. *Image and vision computing*, 21(11): 977–1000, 2003. doi: 10.1016/S0262-8856(03)00137-9. [p71]

Yicheng Kang

Miami University

Department of Information Systems and Analytics

Oxford OH, USA

<https://miamioh.edu/fsb/directory/?up=/directory/kangy10>

ORCID: 0000-0002-8512-2722

kangy10@miamioh.edu